

HiTEA-3D: A Design Framework for High-Throughput and Energy-Aware 3D CNN Acceleration on FPGA

Fatima Hameed Khan , *Student Member, IEEE*, Muhammad Adeel Pasha , *Senior Member, IEEE*, and Shahid Masud , *Senior Member, IEEE*

Abstract—This paper introduces HiTEA-3D, a design framework for High-Throughput Energy-Aware acceleration of 3D Convolutional Neural Networks (CNNs) in resource-constrained environments such as Field Programmable Gate Arrays (FPGA), through mixed-precision quantization, dynamic tiling, and multiple data fetching modes. The HiTEA-3D framework accepts a PyTorch-described model and FPGA specifications as input and generates a synthesizable Verilog code for implementation. The resulting FPGA based design employs a 3D systolic array architecture to maximize the data reuse across spatio-temporal dimensions. The size of the 3D systolic array is determined by minimizing latency while effectively utilizing the available FPGA resources. The basic Processing Element (PE) block used for computing is designed to support variable word-lengths for both the input feature maps and the kernel blocks. The proposed framework has been evaluated on three popular 3D CNN models, C3D, I3D, and R(2+1)D, and implemented on Xilinx Virtex-7 and Zynq UltraScale FPGAs. The experimental results have shown that the hardware accelerators generated through the HiTEA-3D framework deliver improved performance, achieving an average increase of 50% in throughput and a 33% reduction in energy consumption across all tested networks. The hardware architecture is parameterized and can be ported to various implementation platforms. The designs can be quickly produced by specifying high-level specifications of CNN models and FPGA resources.

Index Terms—Design automation framework, 3D CNN, FPGA, hardware architecture, circuits and systems for AI.

I. INTRODUCTION

CONVOLUTIONAL Neural Networks (CNNs) are among the state-of-the-art artificial intelligence(AI) algorithms for image-based tasks [1]. However, the growing focus on

video-related applications [2], such as video surveillance, autonomous driving, and action recognition, demands processing temporal frames of volumetric data. In this context, 3D CNNs have gained attention for capturing temporal features from video frames without relying on specialized time-domain methods like LSTMs. 3D CNNs have also demonstrated superior performance in video classification, particularly in tasks such as Human Action Recognition (HAR). The additional dimension in 3D CNNs increases data handling complexity, on-chip memory requirements, and computational throughput when compared to the conventional 2D CNNs. Hence, it is a significant challenge to map these large-scale 3D models onto edge devices, where fast and accurate processing must be achieved using limited resources and minimal energy consumption.

From a hardware perspective, FPGAs have recently gained significant traction in AI and machine learning (ML) applications [3]. Their flexibility in handling custom architectures like CNNs and transformers, combined with lower power consumption than GPUs, makes them attractive for edge AI solutions. Additionally, major cloud providers such as Amazon Web Services (AWS) [4] and Microsoft Azure [5] have integrated FPGAs into their cloud offerings, as it allows users to deploy customizable hardware acceleration for compute-intensive applications. Previously, several FPGA-based accelerators have also been developed for 3D CNNs [6], [7], [8], [9], [10], [11], [12], but their design space exploration is often limited to a single and older 3D model, such as C3D [13]. In addition, manually designing custom FPGA accelerators for individual models poses significant challenges in terms of complexity and designer productivity. To address these issues, this paper presents a design automation framework, HiTEA-3D, that yields a high-throughput and energy-efficient FPGA-based hardware architecture for 3D CNNs that simplifies design space exploration for key hardware parameters and is scalable to accommodate any type of 3D CNN model.

Some design automation approaches that can map 2D CNNs onto FPGA platforms have been reported using various optimization techniques [14], [15]. However, these techniques primarily focus on the spatial dimensions, without considering the temporal planes and data fetching complexities across multiple dimensions. Some distinguishing features of the proposed HiTEA-3D include mixed-precision quantization,

Received 12 February 2025; revised 25 May 2025 and 27 July 2025; accepted 17 August 2025. Date of publication 1 September 2025; date of current version 26 February 2026. The review of this article was arranged by Associate Editor Alejandro Linares-Barranco. (Corresponding author: Fatima Hameed Khan.)

The authors are with the Department of Electrical Engineering, Syed Babar Ali School of Science and Engineering, Lahore University of Management Sciences, DHA, Lahore 54792, Pakistan (e-mail: fatima.k@lums.edu.pk; adeel.pasha@lums.edu.pk; smasud@lums.edu.pk).

Digital Object Identifier 10.1109/TCASAI.2025.3604383

multi-objective optimization of resources and performance, and dynamic data tiling that result in memory footprint reduction and efficient data handling in the temporal dimension. Here, dynamic data tiling refers to specifying different tile sizes for each layer. To the best of our knowledge, only two automation frameworks for 3D CNN accelerators [16], [17] have been reported where the focus is limited to single parameter optimization with no regard for word-length precision or data tiling.

Another major contribution of the proposed HiTEA-3D framework is the implementation of 3D convolution operations on a 3D systolic architecture to exploit data reuse in the temporal dimension. The previous 3D CNN hardware accelerators [6], [7], [8], [9], [10], [11], [12] have addressed the processing of temporal data by converting 3D convolutions into multiple 2D convolutions, that is then processed on separate 2D Processing Element (PE) arrays resulting in more on-chip memory and FPGA resources. However, the hardware generated by HiTEA-3D mitigates this issue by implementing a 3D systolic architecture. Moreover, the basic PE block of the 3D systolic array is designed to support mixed-precision models. The connections of the PE block are configured at the time of instantiation to support varying word-lengths for different layers. The size of the 3D array is determined by analyzing the impact of latency on each dimension. Based on the resulting parameters, the synthesizable Verilog code for energy-efficient and high throughput FPGA accelerator for 3D CNN is generated.

HiTEA-3D has been evaluated on three widely used 3D CNN models, C3D [13], I3D [19] and R(2+1)D [20], across different FPGA devices. Performance metrics demonstrate that the generated hardware exceeds previous FPGA-based designs either generated through 3D CNN tool flows or produced manually. The major contributions of this work are summarized as follows,

- A design automation framework for 3D CNN has been developed that efficiently maps 3D CNNs on an FPGA platform while optimizing key hardware parameters.
- Compact representation of large 3D CNNs models has been obtained using mixed-precision quantization where the quantization policy balances the trade-offs between hardware resources and classification accuracy.
- Off-chip memory access has been reduced by exploring dynamic data tiling and various data fetching modes.
- An efficient 3D systolic array architecture with associated control flow for 3D convolutions has been proposed to maximize data reuse in both the spatial and temporal dimensions.
- A reconfigurable PE block has been designed that supports variable word-lengths to enable mixed-precision computations across different layers of the 3D CNN.

The remainder of the paper is organized as follows. Section II reviews recent FPGA-based 3D CNN architectures and design automation frameworks. Section III provides the details of the mixed-precision quantization scheme, while Section IV outlines the proposed memory access optimization strategies. Section V then focuses on the internal architecture of the FPGA-based hardware accelerator. Section VI discusses the results

obtained from the FPGA designs, generated through the proposed automation tool for various 3D CNN models and compares them with the existing 3D CNN accelerators. Finally, Section VII concludes the paper.

II. LITERATURE REVIEW

While 3D CNNs have been extensively studied, only a few efforts have focused on accelerating these networks on FPGA based hardware platforms. Most prior research has concentrated on older 3D CNN models, such as C3D, which has now been surpassed in accuracy by more recent models, I3D [19], R(2+1)D [20], etc. From an implementation perspective, the architecture in F-C3D [8] comprises multiple parallel 2D kernel blocks that are further extended into E3DNet [9] based on a 3D-1 bottleneck residual block and sliding window approach. Liu et al. [10] have proposed a hardware design to accelerate both 2D and 3D CNNs by incorporating a customized mapping module to eliminate large feature matrices storage. Shen et al. [11] have introduced a template-based architecture that utilizes the Winograd algorithm for accelerating both 2D and 3D CNNs and provided an analytical model for rapid design space exploration. All of these studies have evaluated their architectures solely on the C3D model. Sun et al. [12] have introduced a hardware-aware pruning algorithm for 3D CNNs on FPGAs and their evaluation included both C3D and R(2+1)D models. Tian et al. [7] have optimized off-chip memory access through different loop ordering strategies and proposed non-overlapping tiling configurations for 3D CNNs. In contrast, the authors in [6] have explored various design parameters to accelerate the I3D network. Here, the hardware complexity is reduced by applying fixed quantization, while the frequency of DRAM access is minimized by optimizing the data tile sizes. Toupas et al. [21] have also applied quantization and optimized the mapping of the advanced X3D model onto FPGA platforms. However, these prior studies are confined to a particular 3D model and explore limited design parameters. This highlights the need for a comprehensive design automation framework that not only rapidly generates 3D CNN hardware architectures, but also boosts the computational efficiency of inference in resource-constrained environments.

To the best of our knowledge, HARFLOW3D [16] and fpgaHART [17] are the only tool flows that automate the design of hardware architectures for 3D CNNs. HARFLOW3D [16] formulates mathematical models for latency and resource consumption and optimizes them using simulated annealing to minimize inference latency. In contrast, fpgaHART [17] optimizes the throughput of the accelerator for given resource constraints and network characteristics. Our proposed framework, HiTEA-3D, distinguishes by optimizing multiple design parameters to create an efficient accelerator for 3D CNNs. The overall system throughput has been increased by applying mixed-precision quantization and implementing convolution operations on a new 3D systolic architecture. Energy efficiency is further enhanced by reducing on-chip and off-chip memory traffic through dynamic tile size configuration and exploring various data fetching modes.

TABLE I
SUMMARY AND COMPARISON OF RELATED LITERATURE REVIEW

Work	[8]	[9]	[10]	[11]	[12]	[7]	[6]	[22]	[16]	[17]	HiTEA-3D
Design Approach	Handcrafted								Framework-Generated		
Model Compression	None	Archi***	None	None	Prun**	None	+Quant	None	None	None	+Quant
Precision	Fixed	Fixed	Fixed	Fixed	Fixed	Fixed	Fixed	Fixed	Fixed	Fixed	Mixed
DRAM Access Optimization	No	No	Yes	Yes	No	Yes	Yes	Yes	No	No	Yes
Tiling Configuration	Uni*	Uni*	Uni*	Uni*	Uni*	Uni*	Uni*	Uni*	Uni*	Uni*	Dynamic
Model Scalability	No	No	No	No	No	No	No	No	Yes	Yes	Yes
Hardware Architecture	2D Array	SW ¹	2D Array	SW ¹	2D Array	2D Array	2D Array	2D Array	SW ¹	SW ¹	3D Array
PE Size Exploration	No	No	No	No	No	Yes	Yes	No	No	No	Yes

*Uniform Tiling Configuration, **Pruning, ***Architectural Changes, +Quantization ¹Sliding Window

Hence, the HiTEA-3D ensures optimized performance for compute-intensive 3D CNNs in resource-constrained environments.

As far as data tiling configuration is concerned, some publications have explored limited tiling dimensions [8], [11] where the remaining tiling parameters are either derived from them or left unattended. Such approaches are not suitable for larger networks and may result in sub-optimal tiling configurations. Sun et al. [12] proposed a 3D CNN accelerator for C3D and R(2+1)D networks where they have presented only two fixed-size tiling configurations. The authors in [6], [7] adopt fixed-size tiling configuration across the network and evaluate their results on single models where [7] focuses on the C3D and [6] on the I3D. In another work [22], various data fetching modes have been explored for fixed-size tiling configuration. These prior approaches rely on fixed tile sizes limiting their flexibility. Our proposed framework, HiTEA-3D, overcomes these limitations by introducing a dynamic tile configuration scheme that assigns different tile sizes to each layer.

Furthermore, all these FPGA based 3D CNN designs address temporal data reuse by decomposing 3D convolutions into multiple 2D convolutions that are processed across different PE arrays. However, this results in excessive data transfers and increased energy consumption. In contrast, our paper introduces an energy-efficient FPGA accelerator that supports temporal data reuse through a 3D systolic array. Wang et al. [18] introduced the concept of a systolic cube with an output stationary dataflow, which limits data reuse and parallelism. In comparison, our accelerator improves data locality by implementing a weight stationary dataflow that enables scalable parallelism and improves the throughput.

Table I summarizes related FPGA-based hardware accelerators for 3D CNNs and compares them with our proposed approach. Unlike previous methods that rely on fixed-size tiling and specific models, HiTEA-3D supports dynamic tile configuration, mixed-precision quantization, and flexible hardware design. It optimizes computational efficiency across different layers while minimizing memory access and energy consumption. This comprehensive approach makes it highly suitable for resource-constrained environments.

The overall architecture of HiTEA-3D comprises three distinct modules that deals with i) hardware-aware mixed-precision quantization, ii) memory access optimization, and iii) FPGA-based hardware architecture generation, as shown in Fig. 1. It takes as input a pre-trained CNN model, input

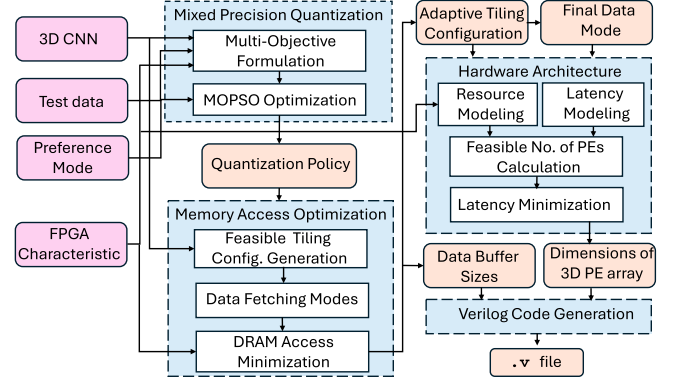


Fig. 1. Overall architecture of the proposed framework, HiTEA-3D.

test data, application preference modes, and the target FPGA device parameters. The computational complexity of 3D CNNs has been reduced by compressing the model through the proposed hardware-aware mixed-precision quantization. The final quantization policy is determined by optimizing two conflicting objective parameters, i) resource usage, and ii) classification accuracy through the Multi-Objective Particle Swarm Optimization (MOPSO) algorithm. Subsequently, HiTEA-3D minimizes off-chip memory access by adjusting dynamic data tiling configurations and exploring various data-fetching strategies. Based on these observations, it evaluates the size of the 3D systolic array architecture and generates a synthesizable code for the optimized hardware architecture. Simultaneously optimizing the computational complexity and the FPGA resources, HiTEA-3D ensures efficient 3D CNN inference across a wide range of FPGA platforms. The different parts of HiTEA-3D are discussed in detail below.

III. MIXED-PRECISION QUANTIZATION

In the proposed framework, HiTEA-3D, a symmetric post-training quantization is applied to compress both weights and activations in each layer while balancing trade-offs between the selected objective parameters. A custom quantization scheme, outlined in [25], has been employed as,

$$Q(r, b) = \text{int} \left(\frac{r}{S} \right) - Z \quad (1)$$

Where Q represents the quantization function applied to the 32-bit floating-point (FP) input tensor, r . The parameter

b denotes the number of bits used for quantization, while S is a real-valued scaling factor used to map FP values into the quantized range. Finally, Z is the zero-point, which maps the exact quantized value for 0 in r . The $\text{int}(\cdot)$ is an integer function that rounds off to the nearest whole number. Performance degradation is the primary cost function associated with the quantization process, and the HiTEA-3D framework aims to minimize the accuracy loss while reducing resource utilization. The step-by-step functionality of the hardware-aware mixed-precision quantization algorithm is outlined in Algorithm 1. Each layer is treated independently in the proposed algorithm to reduce the search space for finding the feasible word-length for that specific layer. Given a pretrained 3D CNN with L layers, the recommended quantization policy, which is the output of Algorithm 1, is defined as the word-lengths for weights, b_w^l , and activations, b_a^l , for $\forall l \in L$ layers. Considering the inherent trade-offs, our work focuses on two key objective functions. The first is the inference error, and the other is the resource utilization. The multi-objective hardware-aware mixed-precision quantization problem can be formulated as,

$$\min F^l(x) = \{f_1^l(x), f_2^l(x)\},$$

$$\text{where } x = [Q(\theta, b_w^l), Q(\alpha, b_a^l)] \text{ s.t. } b_w^l, b_a^l \in B. \quad (2)$$

where B represents the set of candidate bit-lengths used as the search space in HiTEA-3D. The search space for quantization includes multiple-of-two values between 2 and 8 bits, as these word-lengths are supported by the proposed hardware architecture. $F^l(x)$ corresponds to the objective function formation for the l^{th} layer. θ is the set of trained parameters that is quantized to b_w^l bit-length. α is the set of activation feature maps for the l^{th} layer. Our optimization problem explores a 2-dimensional search space, where bit-lengths for activations and bit-lengths for weights are considered for each layer. The solution converges in a 2-dimensional objective space, representing the trade-offs between accuracy and resource utilization. A detailed formulation of the two objective functions used in HiTEA-3D is provided below.

A. Objective Function Formulation

Our proposed approach estimates resource utilization by incorporating its mathematical model directly into the optimization loop. Empirical measurements for this objective formation are gathered from FPGA Lookup Table (LUT) utilization for a Multiplier-Accumulator (MAC) block. The LUT requirement for various MAC blocks, for different bit-lengths of weights and activations from the search space B , is formulated as,

$$LUT_{MAC} \approx \left(\underbrace{\left\lceil \frac{2^4 \times 3}{2^{LUT_{type}}} \right\rceil}_{\text{Addition}} + \underbrace{\left\lceil \frac{2^{max(b_a^l, b_w^l)}}{2^{LUT_{type}}} \right\rceil}_{\text{Multiplication}} \right) \times (b_a^l + b_w^l) \quad (3)$$

Where LUT_{type} represents the number of inputs for the LUTs in the targeted FPGA. The expression for LUT utilization for the adders and multipliers in a MAC block is leveraged from

Algorithm 1 Hardware-Aware Mixed-Precision Quantization

Require: LUT input pins (LUT_{type}), List of network layers (L), Preference mode (P_m), Dataset (D)

Ensure: Word-length for each layer (WL)

```

1: Define:  $MOP2(x, l^{\text{th}} \text{ layer}, (l-1)^{\text{th}} \text{ output})$ :
2:    $Z_1 = \text{evaluate\_resource\_utilization}(x)$ 
3:    $Q = \text{Quant\_Layer}(l^{\text{th}} \text{ layer}, (l-1)^{\text{th}} \text{ output}, x)$ 
4:    $\text{Prediction} = L[l+1 : \text{end}](Q)$ 
5:    $Z_2 = \text{MSE}(\text{Prediction}, \text{Target})$ 
6:   return  $Z_1, Z_2$ 
7:  $L = \text{len}(\text{layers})$ 
8: for  $i = 0$  to  $L - 1$  do
9:   if  $i == 0$  then
10:     $Z(x) = MOP2(x, L[i], D)$ 
11:   else
12:     $\text{prev\_out} = \text{Quant\_Layer}(L[0 : i - 1], D, WL[0 : i])$ 
13:     $Z(x) = MOP2(x, L[i], \text{prev\_out})$ 
14:   end if
15:    $PS = \text{MOPSO}(Z(x))$  # MOPSO Algorithm
16:   if  $P_m == \text{'accuracy'}$  then
17:     $WL[i] = PS$  (where  $PS.\text{inf\_error}$  is the smallest)
18:   else if  $P_m == \text{'resource utilization'}$  then
19:     $WL[i] = PS$  (where  $PS.RU$  is smaller than  $RU$  for the
        lowest accuracy)
20:   end if
21: end for

```

[23] and [24], respectively. These approximations are used to formulate the objective function for resource estimation. Other factors, such as resource sharing and optimization, may also affect the actual LUT utilization.

Our quantization approach reduces the bit precision of the network parameters while minimizing accuracy loss. Although each layer has been evaluated independently for hardware resource estimation, the overall network accuracy is measured as a whole. The optimization of each layer considers the effect of quantization of the previous layer, thus allowing the overall process to factor in and mitigate the error accumulation across the network. The mixed-precision quantization strategy has been designed by analyzing the contribution of each layer to the overall performance of the input 3D CNN model. Each layer of the network is connected as follows,

$$Y = W_L (W_{L-1} \dots (W_1 X_1)) = W_L X_L \quad (4)$$

Where W_L and X_L represent the tensor of weights and input activation for l^{th} layer, respectively. The recursive nature of the initial layers amplifies their impact on the overall accuracy. This pattern is also observed in the Mean Squared Error (MSE) for each layer. The objective function for inference error is defined as follows,

$$MSE_l = \|Y - \hat{Y}_l\|_F^2 \quad (5)$$

$$\hat{Y}_l = W_L (X_L \dots Q(W_l, b_w^l) (Q(\hat{X}_l, b_a^l))) \quad (6)$$

Where $\|\cdot\|_F$ is the Frobenius norm and $\hat{X}_l$ is the quantized output of $l-1$ layer for all layers except the first layer. For

$l = 1$, $\hat{X}_l$ represents the input given to the network. In mixed-precision approach, quantization is not uniform across data types as different bit-lengths are selected for weights and input activations based on their range and sensitivity to the quantization error. In the proposed framework, the mixed-precision optimization problem has been structured around the two objective functions, and the solution has been derived using a modified MOPSO algorithm. The next subsection provides a detailed explanation of the bit allocation process using our modified MOPSO algorithm.

B. Bit Allocation Optimization Algorithm

The Particle Swarm Optimization (PSO) algorithm is a stochastic, population-based optimization technique where a set number of particles explore the search space to find an optimal solution. The multi-objective PSO algorithm [26], known as MOPSO, has been chosen to find a solution for our bit allocation problem due to its faster convergence rate and fewer input parameters compared to other popular optimization methods [27]. MOPSO evaluates a population of candidate solutions (representing bit-length selections) based on its own experience and the collective knowledge of the swarm.

Our approach formulates mixed-precision quantization as a minimization problem, thus prioritizing non-dominant particles with the lowest objective values. MOPSO stores the experience of each particle after every iteration in a global repository. This repository is continuously updated based on the objective function values, and each particle selects a leader from the repository to guide its movement toward the global optimum. The particles' positions (bit-values) are then updated using the following equations.

$$v[t+1] = wv[t] + c_1 R_1 (P_{best} - x[t]) + c_2 R_2 (G_{best} - x[t]) \quad (7)$$

$$x[t+1] = x[t] + v[t+1] \quad (8)$$

Where w represents the inertia value that assigns weight to the particle's previous velocity $v[t]$. R_1 and R_2 are random numbers between 0 and 1. P_{best} is the previous best position of the particle and G_{best} is the best position of the leader. c_1 and c_2 are the social learning parameters that depict the weights of the personal and global learning positions, respectively. The social learning parameters and inertia have been adopted from the theoretical study [28], set at $c_1=3$, $c_2=1.5$, and $w=0.9$, respectively. The original MOPSO algorithm does not confine particle positions within predefined boundaries, possibly allowing bit values to exceed the allowed range. To prevent this, HiTEA-3D incorporates a hybrid restriction technique from [28] to ensure in-bound searches.

$$v[t+1] = \begin{cases} -\gamma \left(\frac{v[t+1]}{1 + \left| \frac{v[t+1]}{x_{max} - x[t]} \right|} \right), & v[t+1] > 0 \\ -\gamma \left(\frac{v[t+1]}{1 + \left| \frac{v[t+1]}{x[t] - x_{min}} \right|} \right), & v[t+1] < 0 \end{cases} \quad (9)$$

This modification speeds up MOPSO convergence by keeping particle positions within search space boundaries. MOPSO

iteratively navigates through the objective space and gradually converges to a set of pareto optimal solutions. The final bit-length selection for recommended bit-lengths weights and activations in each layer is based on the input preference parameter. The next section details the methodology for reducing the on-chip memory access frequency.

IV. MEMORY ACCESS OPTIMIZATION

Each layer of the 3D CNN model takes four-dimensional input data ($M \times D \times H \times W$) as input. Here, $H \times W$ represents spatial dimensions and D is the temporal depth. M spatiotemporal feature maps of the input data are required to convolve with the five-dimensional kernel volume ($M \times N \times K_{size}$ where $K_{size} = K_D \times K_H \times K_W$). Their convolution yields N number of three-dimensional output channels. This huge multidimensional data generally does not fit onto the limited on-chip Block RAM (BRAM) of the FPGA. Thus, the acceleration of 3D CNNs requires the tiling of data in all five dimensions ($T_n \times T_m \times T_d \times T_h \times T_w$), resulting in more frequent external DRAM access. The memory access of 8-bit data transfer consumes approximately 200 times more energy than that required for the corresponding 8-bit MAC operations [29]. Thus, the selection of proper tiling parameters for each layer is an important consideration for an efficient implementation of 3D CNNs on FPGA platforms. The data tiling methodology is shown in Fig. 2. It is observed that the tiling in D , H , and W dimensions introduces the overlapped layer of size OL in each dimension where OL is equal to $K_C - S_C$, and K_C and S_C are the filter size and stride in $C \in D, H, W$ dimensions. Repeated loading of overlapped data increases memory transfer demands, making the choice of tiling parameters (T_d, T_h, T_w) crucial in controlling the rate of external memory access. Since each layer has unique data overlap and processing characteristics, a custom tiling configuration is applied throughout the network. The remaining tiling parameters (T_m, T_n) are explored using different tile fetching modes, as shown in Fig. 2. In the Single Input Multiple Output (SIMO) mode, a single feature map is convolved with the T_n number of weight blocks, producing T_n number of partial output blocks. In Multiple Input Single Output (MISO) mode, the T_m input channels are each convolved with corresponding filters, and the partial results for a single output tile are accumulated in on-chip memory. Conversely, Multiple Input, Multiple Output (MIMO) mode simultaneously processes T_m input feature maps to compute T_n output channels. Finally, the All Input, Multiple Output (AIMO) mode eliminates the need for partial results by computing complete outputs for T_n channels in a single pass.

Each tiling parameter is adjusted to minimize memory access based on the network dimensions and the available BRAM size. The viable candidates for each tiling dimension have been determined by identifying the exact even divisors of the layer's dimensions. Then, all combinations of these candidates are evaluated across the four data-fetching modes. The final configuration is selected to minimize DRAM access while keeping BRAM utilization within a feasible range. The calculations for

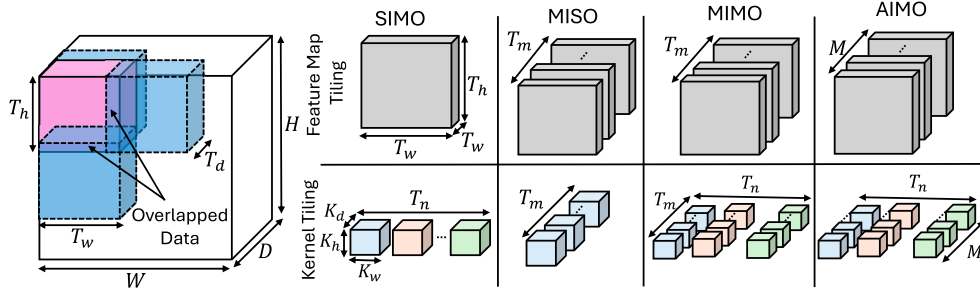


Fig. 2. Tiling configuration for each mode of data fetching mode.

on-chip memory utilization of the output, input, and weight buffers for each layer, l , are given below,

$$BUF_{out} = \left\lceil \frac{T_n \times O_{size} \times b_a}{BRAM_{block}} \right\rceil \quad (10)$$

$$BUF_{in} = \left\lceil \frac{T_m \times T_d \times T_h \times T_w \times b_a}{BRAM_{block}} \right\rceil \quad (11)$$

$$BUF_{weight} = \left\lceil \frac{T_n \times T_m \times K_{size} \times b_w}{BRAM_{block}} \right\rceil \quad (12)$$

Where $BRAM_{block}$ is the size of the BRAM block of the target FPGA and b_a is the word-length for activation feature map. O_{size} is the size of output resulting from the convolution of a single input tile, $O_{size} = O_d \times O_h \times O_w$ and each dimension of the output chunk is calculated as $O_c = \left(\frac{T_c - K_c}{S_c} + 1 \right)$ where $c \in \{d, h, w\}$. There is also a dedicated memory to accumulate partial results which is four times greater in size than the output buffer. The proposed data tiling approach ignores all tiling configurations that exceed the maximum allowable BRAM utilization for a particular FPGA device. The input tile of size $T_m \times T_d \times T_h \times T_w$ is loaded and the total input reads, I_{inp} , from a DRAM is defined as,

$$I_{inp} = T_m \times T_d \times T_h \times T_w \times b_a \quad (13)$$

Along with the input tile, its corresponding kernel blocks are also fetched for T_n output feature maps. It requires the I_{wgt} number of reads from a DRAM which is calculated as,

$$I_{wgt} = T_n \times T_m \times K_{size} \times b_w \quad (14)$$

Where b_w represent the bit-length for the kernel block. The convolution of these input tiles and kernel blocks will result in T_n partial output tile of size O_{size} . The I_{comp} read requests are made to an external DRAM for complete calculation of the output tile,

$$I_{comp} = (I_{inp} + I_{wgt}) \times \left\lceil \frac{M}{T_m} \right\rceil \quad (15)$$

The kernel block will slide in all three dimensions of the input feature maps and it will be repeated for all the output feature maps. Thus, the total DRAM read requests, I , to cover all output feature maps for each layer are defined as,

$$I = iter_n \times I_{comp} \times iter_h \times iter_w \times iter_d \quad (16)$$

$$iter_c = \begin{cases} 1 + \left\lceil \frac{H - T_c}{T_c - K_c + 1} \right\rceil, & c \in \{h, w, d\} \\ \left\lceil \frac{N}{T_c} \right\rceil, & c \in \{m, n\} \end{cases} \quad (17)$$

At each iteration, an output tile of size, O_{size} , is generated and written to the external DRAM. The total number of write requests, Wr , is defined as,

$$Wr = O_{size} \times iter_h \times iter_w \times iter_d \times iter_n \times b_a \quad (18)$$

The total number of DRAM accesses required for processing a tile is calculated as the summation of I and Wr . HiTEA-3D iterates over all network layers to identify the most feasible tiling configuration that minimizes DRAM access. This selected configuration is then used for further evaluation and hardware architecture design. The proposed data tiling methodology has been designed to map all layers on the same design through pipelining and parallel processing. The next section explains the corresponding architecture in detail.

V. FPGA-BASED HARDWARE ACCELERATOR

This section introduces the hardware architecture design for HiTEA-3D by discussing each functional block and its role in efficient processing of 3D CNNs. The overall architecture is shown in Fig. 3. It begins with the data scheduler, that fetches data from the external DRAM and loads it into the designated on-chip buffers according to the selected fetching mode. Once in the buffer, the data distributor block directs the transformed data to the 3D systolic array. The distribution of incoming data is synchronized across multiple processing arrays. This scheduling and distribution process minimizes the

$$Lat_{3D_array} = \underbrace{(PE_z + (PE_x \times PE_y - 1))}_{\text{cycles for first pixel}} \times \underbrace{\frac{2 \times O_{size}}{PE_x \times PE_y}}_{\text{factor for all pixels}} \times \left(\underbrace{3}_{\text{3 steps for each PE}} + \underbrace{1}_{\text{data transfer}} \right) \quad (19)$$

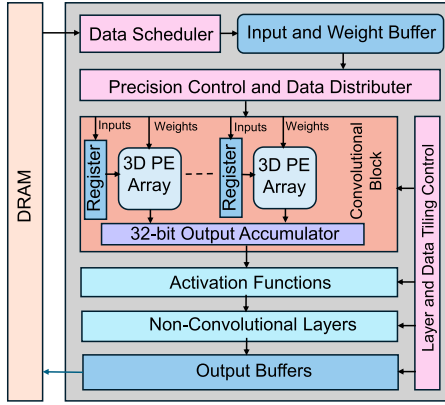


Fig. 3. The overall architecture of the 3D hardware accelerator.

data movement and allows parallelized processing within the systolic array. The 3D systolic array performs core convolution operations by leveraging maximum data reuse across spatial and temporal dimensions. Once fetched from the buffer, the input data is fully utilized in the 3D PE array for the given tile. The weights are kept stationary in the PE block during the processing of the tile. After convolution, the layer control unit routes the output data to the activation unit. Subsequently, the data is forwarded for any additional post-convolutional operations specified by the network model. Both the activation and non-convolutional hardware blocks work in a pipelined manner to process batches of data synchronously. The output buffer stage manages data flow through subsequent network layers and partial accumulation before transferring the data back to the external DRAM. This process is repeated tile by tile until the entire dataset has been processed.

The dimensions of the 3D systolic array are based on a tradeoff between computational throughput and resource availability on the FPGA. The configuration begins by analyzing the network layer's dimensions, including the spatial (height and width) and temporal (depth) components of the input feature maps. The optimal systolic array size is determined by considering the workload required to perform 3D convolutions while minimizing the data transfers. This includes examining the spatial extent of each filter, the temporal length required for processing, and the need to maximize data reuse within the array. By matching the dimensions of the systolic array to these characteristics, the architecture ensures that each PE handles a manageable portion of the data, thereby reducing latency and improving energy efficiency. FPGA resource constraints such as on-chip memory capacity, number of LUTs, etc. are considered, allowing the systolic array dimensions to be adjusted for efficient resource usage without exceeding the hardware limits. The next subsection describes the reconfigurable PE block design, that supports variable bit operations and accommodates 3D data flow for adaptive precision and flexibility across different layers.

A. 3D Systolic Array Architecture

The proposed 3D systolic array architecture fully utilizes the inter-array data reusability and reduces the memory traffic

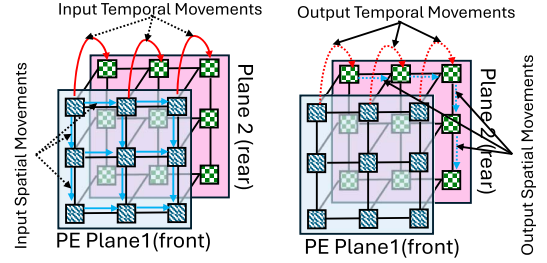


Fig. 4. Connection and dataflow of PEs in 3D systolic array architecture.

due to the overlapped data. It consists of three-dimensional connection of PEs. Each PE is connected to the PEs directly to its right and underneath, facilitating efficient data flow in the spatial dimension, as illustrated in Fig. 4. Each 2D plane of PEs is connected to its corresponding PEs in the rear plane to manage data flow in the temporal dimension. This inter-plane connectivity ensures that the temporal data dependencies are efficiently handled. Fig. 4 illustrates the input data flow and the subsequent generation of output data within the proposed 3D systolic array architecture.

This targeted distribution allows parallel processing of multiple PE blocks to maximize the throughput. In the 3D systolic array, the temporal blocks move conditionally along each direction based on the allocated time schedule and the data availability. An input block is passed to the rear plane every $D - K_D + 1$ clock cycles. When a new data block reaches a PE, that PE does not receive data from its top and left neighboring PEs but instead forwards the incoming data to PE on the right. The rear planes only handle the temporal movement as shown earlier in Fig. 4. In this way, both the input data and the weights are fully utilized in the 3D PE array. This structured data flow enhances the parallel processing capabilities of the systolic array. It also improves the overall energy efficiency by reducing the frequency of on-chip memory accesses.

The proposed 3D systolic array is designed to support variable bit-length operations. The concept of accelerating mixed-precision designs on a single hardware accelerator is inspired by the 2D BitFusion architecture [30]. However, in our work the design of PE block has been adapted to support data flow in multiple dimensions to enable 3D operations. The detailed architecture of a PE block in the proposed accelerator is shown in Fig. 5. The PE architecture comprises of multiple sub-units termed as Bit Bricks (BB) and reconfiguration of these BBs allows variable bit-width operations within a single PE block. Here, each BB performs the multiplication of two 2-bit operands. Additionally, the PE block is equipped with left-shift units and a series of adders to accumulate the results from different BBs. The parameterized left-shift operations allow both low and high-precision operations because the number of shifts in the result of BB is dependent on the bit-length for activations and weights. The results of all BBs of a PE block and the incoming partial sums are stored in a 16-bit register. The resulting partial outputs are transferred to the rear PE plane for reuse in the temporal dimension. Each PE includes a control

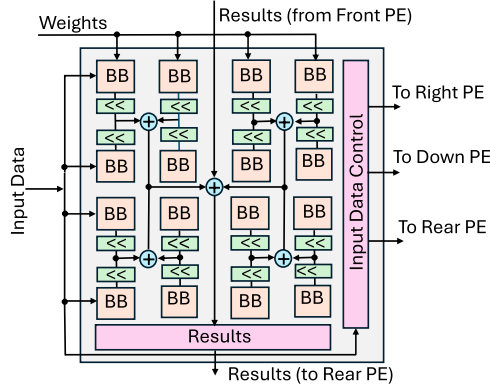


Fig. 5. The overall architecture of the reconfigurable PE module.

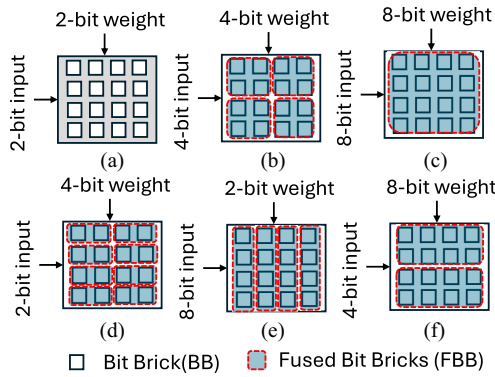


Fig. 6. Various configurations of the proposed PE block for different word-lengths of activation and weight blocks.

logic to synchronize the flow of input and partial output data across different dimensions.

B. Mixed-Precision Support

The proposed architecture supports mixed-precision by allowing each PE to dynamically adapt to different bit-widths for weights and activations based on the requirements of each layer in a 3D CNN. As shown in Fig. 6, all BBs in a PE block are configured at instantiation to form Fused BB (FBB) to handle operands of variable bit-lengths. In Fig. 6(a), no fusion is necessary when processing 2-bit weights and activations, as each BB can independently handle 2-bit multiplications. This setup enables the system to achieve its highest level of parallelism by allowing 16 simultaneous MAC operations. On the other hand, in Fig. 6(b) four BBs are fused to process 4-bit weights and activations allowing 4 simultaneous MAC operations. Similarly, the BB fusion configurations for other variable bit-widths of weights and activations are illustrated in Fig. 6(c)–(f). This adaptable mechanism within each PE block maximizes parallelism and efficiently handles multiple bit-widths required by different CNN layers.

Fig. 7 provides an example of how operations with different bit-lengths are mapped onto the same PE block. Fig. 7(a) illustrates the multiplication of a 4-bit input value 9, and a 4-bit weight value 6, that results in an output value 54. This

operation is decomposed, as shown by the different-colored parallelograms in the figure, and mapped to specific BBs. Each BB block performs a multiplication operation, followed by shifting the results by a specified amount. The final output is obtained by adding these shifted values together. The same PE block is used to map two multiplication operations for values (9x3) and (12x2) involving 4-bit inputs and 2-bit weights, as depicted in Fig. 7(b). Since different activation maps are accumulated after convolution operation, the 4-bit inputs and corresponding 2-bit weights from the first input feature map are processed in the green and pink BBs, while data from the second feature map are processed in the lower blue and red BBs. In Fig. 7(b), same PE block performs two MAC operations, resulting in doubling the throughput compared to Fig. 7(a).

The variable precision increases computational performance by accommodating more operations per PE block. After finalizing implementation parameters like quantization policy, dynamic tiling configuration, and data tiling mode, HiTEA-3D determines the suitable size of the 3D PE array.

C. 3D PE Array Size Exploration

The size of the 3D systolic array plays a crucial role in determining its overall latency and the degree of parallel processing. A smaller PE array size enables a higher number of arrays to fit within the available resources simultaneously processing more output feature maps. In contrast, larger 3D systolic PE arrays allow more input feature maps to be processed simultaneously. However, increasing the size of the PE array reduces the number of such arrays, which in turn limits the output parallelism. The proposed framework, HiTEA-3D, is designed to determine the optimal size of the PE array to minimize latency. The LUT utilization for a single PE block depends on the type of LUT embedded in the specific FPGA device. The LUT utilization of a single PE block was determined offline by synthesizing the PE block on different available LUT types. The resulting value of LUT utilization is used in HiTEA-3D for an accurate estimation of resource requirements for the whole PE array. In CNNs, the convolution operations account for more than 95% of the overall computations [6]. Therefore, the total number of PEs has been evaluated by limiting the framework to configure 80% of the total available LUTs for convolution layers, while leaving the remaining 20% for other essential requirements. Each dimension of the 3D systolic array is bounded such that the total number of PEs does not exceed the available hardware resources. All possible combinations of these dimensions are explored in HiTEA-3D to identify the configuration that achieves the lowest latency. The overall network latency is then calculated using the mathematical model below that represents the processing time of the generated hardware architecture.

The mapping of incoming tile data for each layer is determined by the adopted tile-fetching mode, the bit-length of both the activation maps and the kernel blocks, as well as the tile size and the size of the 3D systolic array. Each array performs simultaneous convolution operations for the corresponding output feature map, while parallelism for the input feature maps is achieved within the PE blocks. The flexible data mapping and

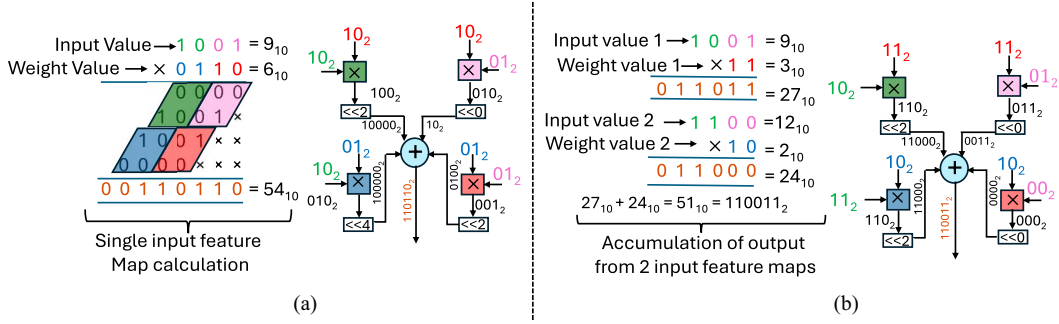


Fig. 7. Mapping of (a) one MAC operation involving 4-bit input and 4-bit weight and (b) two MAC operations involving 4-bit inputs and 2-bit weights on the same PE hardware block.

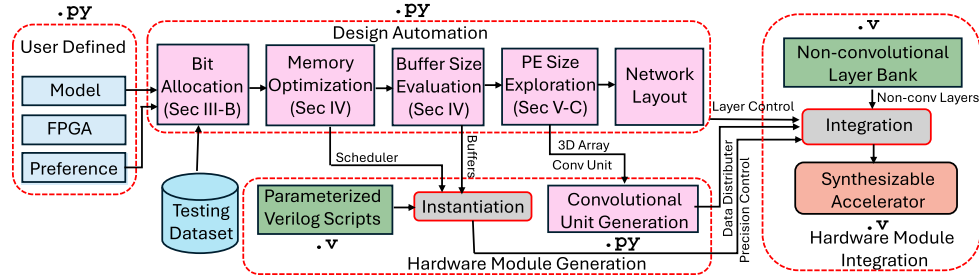


Fig. 8. The proposed tool flow for the generation of FPGA-based hardware accelerators of 3D CNNs.

parallelism ensure that the architecture can process complex 3D CNN workloads with high throughput and minimal latency. The overall processing time, Lat_{tile} of a tile is calculated as,

$$Lat_{tile} = (Lat_{3D_array} \times f_m \times f_n) + Lat_{accum} \quad (20)$$

$$f_n = \left\lceil \frac{T_n}{N_{arrays}} \right\rceil \quad (21)$$

$$f_m = \left\lceil \frac{\left\lceil \frac{T_m}{PE_m} \right\rceil}{M_x \times M_y \times M_z} \right\rceil \quad (22)$$

$$M_c = \begin{cases} \left\lceil \frac{PE_c}{K_c} \right\rceil, & \text{if } \left\lceil \frac{PE_c}{K_c} \right\rceil > 1 \\ 1, & \text{otherwise} \end{cases} \quad \text{s.t. } c \in \{x, y, z\} \quad (23)$$

$$PE_m = \frac{64}{b_a \times b_w} \quad (24)$$

$$Lat_{accum} = (M_x \times M_y \times M_z) + f_m \quad (25)$$

Where f_m and f_n represent the folding factors of T_m and T_n tiles on the given convolution block. Lat_{accum} is the time for accumulation of results for each fold. The overall latency of a layer is defined as the total time required to process all the tiles associated with that layer. The latency is calculated by adding the processing time for each tile. The number of iterations, $iter_c$, required along a specific dimension is described in Eq. (17). In this approach, the dimensions of the 3D systolic array and the number of arrays required to achieve minimum latency are determined for the given model. These optimized parameters are then used to generate the Verilog script for the convolution unit. The next section provides a

detailed explanation of the overall tool flow of the proposed automation framework for the 3D CNNs.

D. HDL Code Generation

HiTEA-3D accepts user input in Python files, including a CNN model described in PyTorch and its test data. The model and data are essential for calculating the inference error during the allocation of bit-lengths to each layer. Fig. 8 illustrates the automatic mapping process of the given 3D CNN model onto the FPGA-based hardware accelerator. The design automation process generates a specific set of parameters at each step to instantiate parameterized Verilog scripts for the fundamental hardware components. For the target 3D CNN, each layer is mapped using a sequence of hardware building blocks designed for its specific functionality. For example, tiling parameters, derived through memory access optimization, are utilized to generate the scheduler module based on the tile-fetching mode. The convolution unit is produced by exploring the optimal 3D PE array size, while the basic PE blocks are instantiated and assembled into the desired array configuration. Predefined non-convolution layers contain Verilog code for common operations such as max pooling, average pooling, normalization, ReLU, and downsampling etc. The network layout is evaluated to determine the sequence of layers required for the complete inference of the model. All these modules are then integrated into a single file, with the overall hardware accelerator controlled by a Finite State Machine (FSM). The integration process includes the configuration of proper connections between the hardware building block modules and the management of the data flow for sequential execution within each layer. Additionally, the

TABLE II
CHARACTERISTICS OF THE EVALUATED 3D MODELS

Models	C3D	I3D	R(2+1)D
Operations in GFLOPs	38.61	50	8.52
No. of Parameters (M)	78.41	12	31.8
Total Layers	27	103	82
No. of Conv Layers	8	24	37

memory system is structured to store kernel weights and feature maps in the external DRAM. The scheduler is designed for efficient transfer of the feature data and the address signals from various modules to the feature buffer. This ensures that the generated hardware is both efficient and tailored to the custom requirements of the given 3D CNN model.

VI. RESULTS AND DISCUSSION

The experimental setup for evaluating the proposed design automation framework, HiTEA-3D, for 3D CNNs has been designed to assess its ability to optimize hardware performance on various FPGA platforms. HiTEA-3D has been tested on three widely recognized and unique 3D CNN models, C3D, I3D, and R(2+1)D, used for video-based applications. The C3D model consists of 8 convolution layers and 3 fully-connected layers. The I3D model is an inception-based architecture built by inflating 2D filters into a third dimension. Compared to C3D, the I3D features a smaller model size yet delivers more accurate results for complex video tasks. Lastly, the R(2+1)D model separates each 3D convolution into distinct spatial and temporal convolutions for efficient feature extraction. Among these, the I3D demonstrates superior performance in capturing detailed temporal dynamics. The overall characteristics of these three models are summarized in Table II.

A. Experimental Setup

The optimization techniques and the design flow for generating Verilog scripts have been developed in Python. HiTEA-3D has been compiled and tested on an Intel Core-i9 laptop running 64-bit Windows 11 having 32 GB RAM. The resulting hardware generated from HiTEA-3D has been synthesized and implemented on two popular Xilinx FPGA platforms, the Zynq UltraScale+ ZCU102 and the Virtex-7 VC709 using Vivado 2020.1. Both ZCU102 and VC709 FPGAs are equipped with ample LUTs, DSP slices, and BRAM resources. The VC709 offers a higher count of these resources, making it better suited for handling large-scale data throughput. Through these varied and advanced FPGA platforms, the adaptability and robustness of the HiTEA-3D framework are demonstrated for different architectures with varying processing needs and memory configurations. Throughput and energy consumption are key parameters for evaluating the performance of a hardware module configured on FPGA platforms. In our work, throughput is measured by calculating the total number of MAC operations performed per second and then normalizing it to the frame count. The energy dissipation of the generated hardware module has been estimated using the empirical method outlined in [29]. Here, the

energy required to process a single data tile, E_{tile} , is calculated as the summation of two components, E_{data} and E_{comp} . The E_{data} represents the energy required for both on-chip and off-chip data traffic obtained from the number of fetches of input features and weights from the memory. The other component E_{comp} , refers to the product of energy consumed by a single PE block and the total number of active PE blocks used to process a data tile.

B. Framework Evaluation

The proposed HiTEA-3D framework was evaluated on three models, C3D, I3D, and R(2+1)D, across two FPGA platforms, ZCU102 and VC709. The results highlight the ability of HiTEA-3D to generate an efficient hardware architecture for 3D CNN while optimizing resource utilization, throughput, and energy consumption. To assess the contributions of different optimizations within the framework, a customized baseline hardware accelerator with the following characteristics was handcrafted: (i) 8-bit fixed precision (ii) a uniform tiling configuration across the network, and (iii) a 2D PE array architecture. The size of the 2D PE array matches that of the 3D PE array, however, the planes are not interconnected. To evaluate the impact of mixed-precision, the baseline model is then modified to implement the resulting quantization policy while retaining the same 2D architectural design and tiling configuration. Similarly, the dynamic tiling configuration model is evaluated by keeping the other two design characteristics consistent with the baseline model, except for the optimized tiling configuration. Likewise, the design incorporating 3D systolic array architecture has the same tiling configuration and 8-bit fixed precision as the baseline model, but its PE planes are interconnected to process the incoming data in the temporal dimension. Finally, all these custom handcrafted designs were compared against the hardware accelerators generated through the proposed HiTEA-3D framework for all three 3D CNN models. Fig. 9 provides a detailed comparison of the key performance parameters among all these different design variants explained above.

The results show reduced latency across all the 3D CNN models due to a smaller memory footprint from lower precision and reduced DRAM access. The 3D systolic architecture further minimizes latency by maximizing data reuse, as explained in Section V-A. The proposed architecture achieves latency reductions of 18%, 56%, and 52% for C3D, I3D, and R(2+1)D, respectively. For throughput considerations, the mixed-precision optimization has the most significant impact on R(2+1)D models due to the peculiar design of the PE architecture. Lower precision enables each PE block to handle multiple MAC operations simultaneously, thus boosting the throughput. In comparison, the dynamic tiling optimization contributes the most to the increased throughput in I3D due to multiple $1 \times 1 \times 1$ convolution layers. These layers allow maximum data to be fetched and processed simultaneously, which results in substantial throughput gains. On the other hand, C3D benefits the most from the 3D systolic architecture, as its layers fully exploit the temporal dataflow in the 3D array. Overall, the HiTEA-3D generated architecture achieves a throughput increase of $2.2 \times$

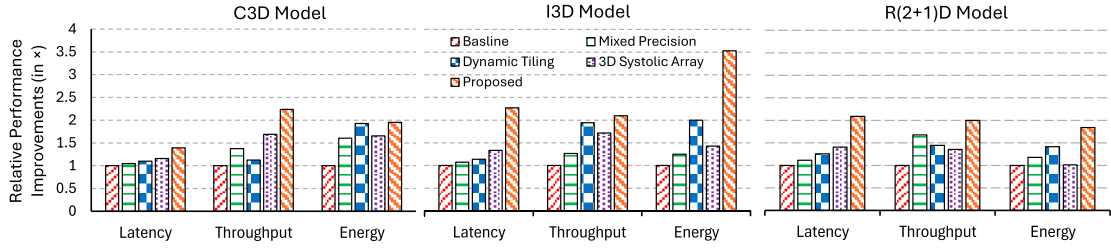


Fig. 9. Performance evaluation and comparison of various hardware accelerators for (a) C3D (b) I3D (c) R(2+1)D models.

TABLE III
PERFORMANCE AND RESOURCE UTILIZATION COMPARISON FOR C3D, I3D,
AND R(2+1)D ON ZCU102 AND VC709 DEVICES

Model	C3D		I3D		R(2+1)D	
FPGA Device	ZCU102	VC709	ZCU102	VC709	ZCU102	VC709
3D PE Array	6×6×6	6×6×6	6×6×6	6×6×6	3×3×3	4×4×4
No. of Arrays	15	25	15	25	126	84
LUT (%)	93	94	92	93	95	90
FF (%)	85	89	89	84	84	86
BRAM (%)	62	59	60	43	52	53
Latency (ms)	85	79	73	44	41	33
Throughput (GOP/s)	693	796	765	960	468	618
Energy (J)	1.00	1.32	1.12	1.19	0.98	1.13

for C3D, $2.1\times$ for I3D, and $2.0\times$ for R(2+1)D. In terms of energy dissipation, the dynamic tiling optimization significantly reduces the energy profile across all networks by minimizing the memory traffic. Since DRAM access is the primary contributor to energy consumption, reducing its rate leads to noticeable energy savings. The proposed HiTEA-3D-generated accelerator leverages these optimizations to achieve energy reductions of 49%, 53%, and 54% for C3D, I3D, and R(2+1)D models, respectively. These results highlight that the HiTEA-3D design automation framework effectively improves computational throughput while reducing the energy dissipation.

Table III demonstrates the flexible nature of the HiTEA-3D framework by adapting the 3D PE array dimensions and array count to the requirements of the target 3D CNN model. For instance, the C3D and I3D model relies on larger $6\times6\times6$ PE arrays with fewer counts while R(2+1)D utilizes smaller arrays with higher array counts. The resource utilization, including LUTs, FFs, and BRAMs, is optimized for each platform, with VC709 generally showing better resource efficiency due to its advanced architecture. The values in Table III exclude DSP utilization because the synthesis was focused primarily on logic resources (LUTs, FFs, and BRAMs). This exclusion was intentional to maintain vendor and device independence for the hardware design generated by the proposed framework. Latency and throughput further reflects the framework capabilities, with I3D achieving the highest throughput of 960 GOP/s on VC709. The energy consumption analysis reveals that ZCU102 consumes slightly less energy due to its relatively smaller hardware architecture. Each evaluated model employs a mixed-precision quantization policy aimed at balancing computational efficiency and accuracy. Regarding the quantization policy of the C3D model, the first five layers use 8-bit precision for both activations and weights. The sixth layer employs mixed precision with 4-bit activations and 8-bit weights, while all subsequent

layers utilize 4-bit precision for both activations and weights. These results demonstrate the HiTEA-3D framework's ability to balance performance metrics while adapting to the unique characteristics of target FPGA platforms and CNN models.

C. Comparison With Previous Reported Accelerators

Table IV compares the HiTEA-3D framework with existing handcrafted designs for various 3D CNNs, including C3D, R(2+1)D, and E3D, implemented on FPGA platforms such as ZC706, VC709, VUS440, and ZCU102. The designs produced through the HiTEA-3D framework demonstrate a significant increase in throughput of 38% on average across all networks compared to the previous custom designs. For instance, the throughput reaches up to 796 GOP/s for C3D on the VC709, surpassing that of the earlier designs, like 334 GOP/s on VC709 reported in [10] and 79 GOP/s on ZCU102 reported in [12]. There is also a significant reduction in latency with the HiTEA-3D generated designs achieving values as low as 79 ms for the C3D model and 33ms for the R(2+1)D model, representing an average reduction of 43%. The energy efficiency comparison shows a substantial improvement of 24% on average across different models. Resource utilization metrics for DSP blocks, LUTs, and BRAM slices further highlight the ability of HiTEA-3D to deliver high performance without exceeding FPGA resource constraints.

Next, a comparison of the efficiency of HiTEA-3D against other automation tool flows, that have been reported for 3D CNN models, is presented. Table V presents a snapshot of this comparison, showing key performance parameters. The throughput of our hardware achieves an average improvement of 63% over previous work, reaching up to 796 GOP/s for the C3D on the VC709 platform, compared to 424 GOP/s in [16] and 131 GOP/s in [17]. For latency reduction, our framework-generated hardware achieves an average reduction of 37% across the networks. In terms of energy efficiency, the HiTEA-3D generated design reduces the consumption per clip by an average of 42%. Finally, on the resource utilization front, the DSP utilization is significantly lower compared to the FPGA designs generated in [16] and [17], and there is also a reduction in the overall resource utilization.

In conclusion, the HiTEA-3D framework optimizes multiple key parameters, including throughput, latency, and energy efficiency, to generate a high-performance 3D CNN hardware accelerator. Unlike previous frameworks that utilize 16-bit FP precision, HiTEA-3D reduces memory footprint through

TABLE IV
THE COMPARISON OF PERFORMANCE AND RESOURCE UTILIZATION BETWEEN HiTEA-3D AND OTHER EXISTING STATE-OF-THE-ART ACCELERATORS

Work	[8]	[9]	[10]	[11]	[12]	[7]	[6]	[22]	HiTEA-3D					
Model	C3D	E3D	C3D	C3D	C3D	R(2+1)D	C3D	I3D	C3D	I3D	R(2+1)D	C3D	I3D	R(2+1)D
FPGA Device	ZC706	Intel	VC709	VC709	VUS440	ZCU102	VC707	VC709	VC709	VC709	VC709	VC709	VC709	VC709
Accuracy on UCF101 (%)	-	85.17	-	-	-	88.66	-	95	-	-	-	82.8	94.4	91.7
Throughput (GOP/s)	71	172	334	431	786	79	111	357	1145	412	634	195	796	960
Frequency (MHz)	172	150	120	150	200	150	150	160	200	200	200	200	200	200
Latency (ms)	543	35.3	115	89	49	487	234	107	96	86	70	41	79	44
Energy/clip (J)	52.6	1.27	1.8	2.24	1.3	3.2	1.6	-	1.4	1.3	1.05	0.93	1.32	1.19
DSP (%)	90	93	99	42	53	48	48	96	98	0	0	0	45	51
LUT (%)	41	45	62	56	8.3	54	54	-	85	72	72	72	64	77
BRAM (%)	86.6	-	26	52	30	100	100	52	79	45	52	61	59	43

TABLE V
THE COMPARISON OF HiTEA-3D WITH THE EXISTING DESIGN AUTOMATION TOOLS FOR 3D CNN ACCELERATION

Work	Harflow3d [16]				fpgaHART [17]		HiTEA-3D			
Model	C3D		R(2+1)D		C3D	R(2+1)D	C3D		R(2+1)D	
FPGA	VC709	ZCU102	VC709	ZCU102	ZCU102	ZCU102	VC709	ZCU102	VC709	ZCU102
Accuracy on UCF101 (%)	83.2	83.2	88.6	88.6	83.2	88.6	82.8	82.8	91.7	91.7
Throughput (GOP/s)	424	393	185	173	131	40	796	693	596	468
Frequency (MHz)	150	200	150	200	160	160	200	200	200	200
Latency (ms)	91	98	46	49	296	216	79	85	33	41
Energy/clip (J)	-	1.72	-	-	-	-	1.32	1.00	1.13	0.98
DSP (%)	97	98	98	97	51	66	45	47	43	46
LUT (%)	-	-	-	-	-	-	64	55	62	59
BRAM (%)	63	72	65	69	91	78	59	62	53	52

mixed-precision quantization. Although the C3D models reported in [16] and [17] achieve slightly higher classification accuracy of 0.4%, our approach delivers $1.9\times$ improvement in hardware throughput. These performance gains are further attributed to the implementation of a 3D systolic architecture for improved temporal reusability and dynamic tile sizing for each layer, reducing off-chip data traffic. The proposed approach not only enhances the performance of 3D CNN acceleration on FPGA in terms of classification accuracy, latency, energy efficiency, and throughput, but also streamlines the hardware design process for complex 3D CNNs. By introducing a design automation framework tailored for 3D CNNs, it efficiently explores critical design parameters and generates the most suitable hardware architecture for a given model and target platform. The traditional hardware design methods for CNN accelerators can take designers several days or weeks. In contrast, HiTEA-3D significantly reduces design effort by automatically generating a suitable 3D CNN hardware architecture within an hour.

VII. CONCLUSION

The proposed design automation framework, HiTEA-3D, provides a comprehensive solution for mapping 3D CNN models onto FPGA architectures with optimized performance in terms of throughput, resource optimization, and energy efficiency. By automating key design parameters of mixed-precision quantization and tiling configuration and by incorporating an efficient 3D systolic array architecture, HiTEA-3D efficiently balances hardware constraints with the throughput requirements. The experimental results have demonstrated an average increase of 50% in the throughput with an average reduction of 33% in the energy consumption across different 3D CNN models. The proposed approach reduces resource

utilization and inference latency without compromising classification accuracy, thus demonstrating a scalable and efficient design framework for 3D CNN inference on resource-constrained FPGA platforms. The HiTEA-3D framework can be extended to other deep learning models, upcoming quantization policies, and FPGA characteristics to generate advanced 3D CNN designs that cater to both performance and hardware requirements.

REFERENCES

- [1] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," *IEEE Int. Conf. Comput. Vis. (ICCV)*, Santiago, Chile, 2015, pp. 1026–1034.
- [2] S. Mittal, "A survey of accelerator architectures for 3D convolution neural networks," *J. Syst. Archit.*, vol. 115, 2022, Art. no. 102041.
- [3] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, "A survey of FPGA-based neural network inference accelerators," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 12, no. 1, pp. 1–26, Mar. 2019.
- [4] "Bringing datacenter-scale hardware-software co-design to the cloud with FireSim and Amazon EC2 F1 instances." Accessed: Nov. 25, 2024. [Online]. Available: <https://aws.amazon.com/blogs/compute/bringing-datacenter-scale-hardware-software-co-design-to-the-cloud-with-firesim-and-amazon-ec2-f1-instances/>
- [5] "Microsoft adds FPGA-powered network accelerator to Azure." Accessed: Nov. 25, 2024. [Online]. Available: https://www.theregister.com/2023/11/21/azure_boost_network_accelerator/
- [6] F. H. Khan, M. A. Pasha, and S. Masud, "Towards designing a hardware accelerator for 3D convolutional neural networks," *Comput. Electr. Eng.*, vol. 105, 2023, Art. no. 108489.
- [7] T. Tian, X. Jin, L. Zhao, X. Wang, J. Wang, and W. Wu, "Exploration of memory access optimization for FPGA-based 3D CNN accelerator," *Design, Automation & Test Eur. Conf. Exhibit. (DATE)*, Grenoble, France, 2020, pp. 1650–1655.
- [8] H. Fan, X. Niu, Q. Liu, and W. Luk, "F-C3D: FPGA-based 3-dimensional convolutional neural network," in *Proc. 27th Int. Conf. Field Programmable Logic Appl. (FPL)*, Ghent, Belgium, 2017, pp. 1–4.
- [9] H. Fan et al., "F-E3D: FPGA-based acceleration of an efficient 3D convolutional neural network for human action recognition," in *Proc. IEEE 30th Int. Conf. Appl. Specific Syst., Architect. Processors (ASAP)*, New York, NY, USA, 2019, pp. 1–8.

- [10] Z. Liu, P. Chow, J. Xu, J. Jiang, Y. Dou, and J. Zhou, "A uniform architecture design for accelerating 2D and 3D CNNs on FPGAs," *Electron. MDPI*, vol. 8, no. 1, 2019, Art. no. 65.
- [11] J. Shen, Y. Huang, Z. Wang, Y. Qiao, M. Wen, and C. Zhang, "Towards a uniform template-based architecture for accelerating 2D and 3D CNNs on FPGA," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*, vol. 8, no. 1, 2018, pp. 97–106.
- [12] M. Sun, P. Zhao, M. Gungor, M. Pedram, M. Leeser, and X. Lin, "3D CNN Acceleration on FPGA using hardware-aware pruning," in *Proc. 57th ACM/IEEE Design Automat. Conf. (DAC)*, San Francisco, CA, USA, 2020, pp. 1–6.
- [13] D. Tran, L. Bourdev, R. Fergus, L. Torresani, and M. Paluri, "Learning spatiotemporal features with 3D convolutional networks," *IEEE Int. Conf. Comput. Vis. (ICCV)*, 2015, pp. 4489–4497.
- [14] S. I. Venieris, A. Kouris, and C.-S. Bouganis, "Toolflows for mapping convolutional neural networks on FPGAs: A survey and future directions," *ACM Comput. Surv.*, vol. 51, no. 3, pp. 1–39, May 2019.
- [15] S. Basalama, A. Sohrabizadeh, J. Wang, L. Guo, and J. Cong, "Flex-CNN: An end-to-end framework for composing CNN accelerators on FPGA," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 16, no. 2, pp. 1–24, Jun. 2023.
- [16] P. Toupas, A. Montgomerie-Corcoran, C.-s. Bouganis, and D. Tzovaras, "Harflow3D: A latency-oriented 3D-CNN accelerator Toolflow for HAR on FPGA devices," in *Proc. Int. Symp. Field-Programmable Custom Comput. Mach. (FCCM)*, 2023, pp. 144–154.
- [17] P. Toupas, C.-S. Bouganis, and D. Tzovaras, "fpgaHART: A Toolflow for throughput-oriented acceleration of 3D CNNs for HAR onto FPGAs," in *Proc. 33rd Int. Conf. Field-Programmable Logic Appl. (FPL)*, Belfast, U.K., 2023, pp. 86–92.
- [18] Y. Wang, H. Li, C. Shi, and X. Li, "Systolic Cube: A spatial 3D CNN accelerator architecture for low power video analysis," in *Proc. 56th ACM/IEEE Design Automat. Conf. (DAC)*, Las Vegas, NV, USA, 2019, pp. 1–6.
- [19] J. Carreira and A. Zisserman, "Quo Vadis, Action Recognition? A new model and the kinetics dataset," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 4724–4733.
- [20] D. Tran, H. Wang, L. Torresani, J. Ray, Y. LeCun, and M. Paluri, "A closer look at spatiotemporal convolutions for action recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 6450–6459.
- [21] P. Toupas, C.-S. Bouganis, and D. Tzovaras, "FMM-X3D: FPGA-based modeling and mapping of X3D for human action recognition," in *Proc. IEEE 34th Int. Conf. Appl. Specific Syst., Architect. Processors (ASAP)*, Porto, Portugal, 2023, pp. 119–126.
- [22] F. H. Khan, M. Adeel Pasha, and S. Masud, "Exploring memory access techniques for efficient FPGA-based 3D CNN accelerator design," in *Proc. IEEE 6th Int. Conf. AI Circuits Syst. (AICAS)*, Abu Dhabi, United Arab Emirates, 2024, pp. 218–222.
- [23] G. Dhanabalan, V. Karutharaja, and M. Sakthimohan, "Realization of resource efficient block RAM based eight bit adder in FPGA," in *Proc. IEEE Int. Conf. Intell. Techn. Control, Optim. Signal Process. (INCOS)*, Tamilnadu, India, 2019, pp. 1–5.
- [24] Y. Xie, Z. Li, D. Diaconu, S. Handagala, M. Leeser, and X. Lin, "LUTMUL: Exceed conventional FPGA roofline limit by LUT-based efficient multiplication for neural network inference," 2024, *arXiv.2411.11852*.
- [25] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.
- [26] C. A. Coello Coello and M. Salazar Lechuga, "MOPSO: A proposal for multiple objective particle swarm optimization," in *Proc. Congr. Evol. Comput. (CEC)*, vol. 2, 2002, pp. 1051–1056.
- [27] E. Büyük, "Pareto-based multiobjective particle swarm optimization: Examples in geophysical modeling," in *Optimisation Algorithms and Swarm Intelligence*, London, UK: IntechOpen, 2021.
- [28] M. Clerc, "Confinements and biases in particle swarm optimisation," 2006. [Online]. Available: <https://hal.science/hal-00122799/>
- [29] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [30] H. Sharma et al., "Bit Fusion: Bit-level dynamically composable architecture for accelerating deep neural network," in *Proc. ACM/IEEE 45th Annu. Int. Symp. Comput. Archit. (ISCA)*, Los Angeles, CA, USA, 2018, pp. 764–775.